Exploring Collaboration between a Language and a Non-Language Agent

Harini S I[1*], Somesh Singh[1,2,3*], Yaman K Singla[1], Rajiv Ratn Shah[2], David Doermann[3], Balaji Krishnamurthy[1]

[1]Adobe Media and Data Science Research [2]IIIT-Delhi [3]SUNY at Buffalo [*]Equal contribution

EMNLP2026 BUDAPEST

Motivation

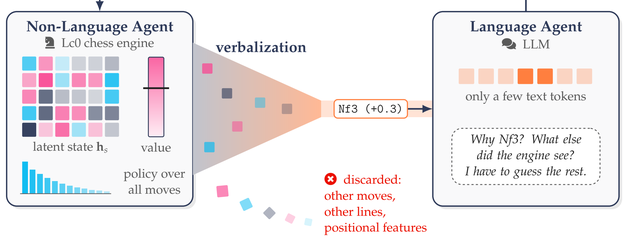
- In many important domains such as robotics and game playing, the strongest available agents are **not language models**. Their expertise lives in internal representations of policy and value (e.g. AlphaZero, RT-1).
- Many useful tasks need both. In chess, an engine plays far better than any LLM but cannot explain a move; an LLM can explain, but cannot play well.

How can an LLM effectively collaborate with a non-language agent?

- We study this in **chess**: strong open engines (Lc0), large public data, and tasks with established metrics.

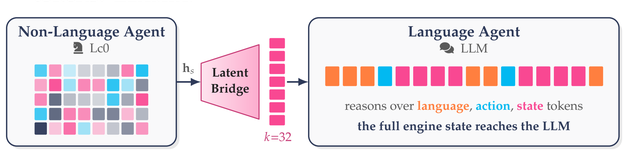
Verbalization Is Lossy

- Current methods call the agent as a tool and **verbalize** its output into text. Only a short summary survives; the rest of its internal representation is lost, at every interaction step.

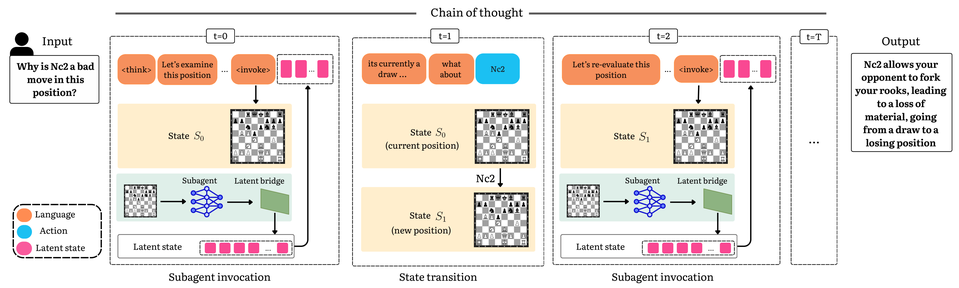


Our Approach

- A **language agent** (Qwen3) collaborates with a frozen **non-language agent** (Lc0-BT4).
- **Latent state internalization:** the engine's internal state enters the LLM as continuous **state tokens**, not text.
- RL trains the LLM to reason over **language**, **action** and **state** tokens: **Llamia**.



Latent State Internalization

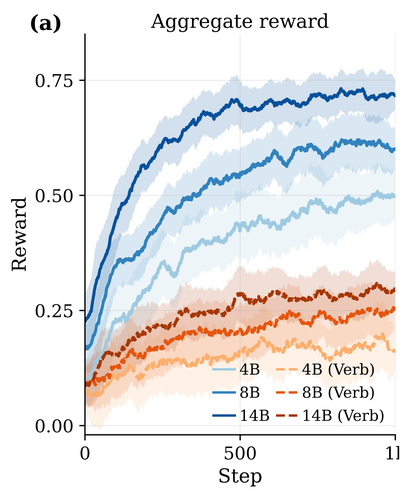


A single chain of thought. The LLM reasons in **language**, calls the engine on the current board, plays an **action** that changes the board, and calls the engine again on the new board.

- **Projection.** Lc0's hidden state $\mathbf{h}_s \in \mathbb{R}^{1024}$ (layer 14 of 15) is mapped by a 3-layer MLP, the **LatentBridge**, to k=32 **state tokens** in the LLM's embedding space: $\mathbf{z}_s = H_\varphi(\mathbf{h}_s) \in \mathbb{R}^{k \times e}$.
- **Dynamic invocation.** Inside its chain of thought, the LLM can play moves that update the board (including hypothetical moves) and call `get_policy` on any position. Each call re-encodes the current state, so the LLM always reasons over the position it is analyzing.
- The LLM decides **when** to call the engine; the call is part of its action space.

Training

- **Stage 1: projector alignment.** Only the LatentBridge is trained, with the LLM frozen, on engine state–policy pairs (predict the engine's move from the **state tokens**).
- **Stage 2: reinforcement learning.** The LLM and LatentBridge are trained jointly with **DAPO**. The reward is each task's metric. **State tokens** are masked from the policy gradient.
- The engine stays frozen throughout.
- **Control:** Llamia-Verb uses the same LLM, engine, data and RL recipe, but receives the engine's output only as text.



Aggregate reward during DAPO training at three LLM sizes. Solid: Llamia (latent); dashed: Llamia-Verb (text).

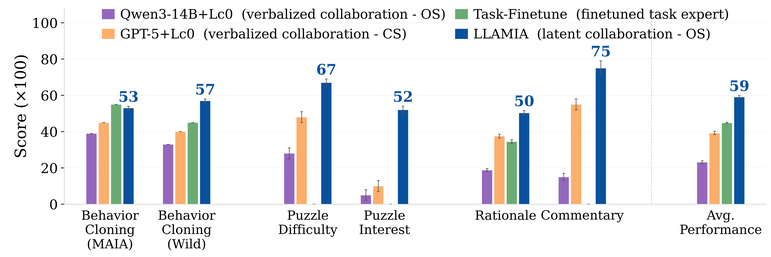
- The method works at **4B, 8B and 14B**; larger LLMs reach higher reward.
- At every size, Llamia trains well above Llamia-Verb, and the gap grows during training, reaching **2–3**× at the end.
- Scaling the LLM does not close the gap: Llamia-4B ends above Llamia-Verb-14B.

Llamia-Bench

A suite of six collaborative chess tasks. None can be solved well by the LLM or the engine alone.

Task	Goal	Metric
Behavior cloning (Maia)	Predict a human's move at a given rating	Move match
Behavior cloning (Wild)	Same, on GM, low-time, rating-gap games	Move match
Puzzle difficulty	Rank puzzles by difficulty	Spearman ρ
Puzzle interest	Rank puzzles by how interesting they are	Spearman ρ
Move rationale	Explain why a move was played	BLEU-2
Game commentary	Narrate a full game (Agadmator-2K, new)	G-Eval

Results



- A single Llamia-14B model is best on all six tasks, ahead of GPT-5 with the same engine as a tool and of task-specific finetuned models.
- On Puzzle Interest, every text-based system scores ≤ 12; Llamia scores **52**.

Further Findings

- **The gain comes from the engine's state.** A raw board encoder or shuffled state tokens give no gain over text.
- **How the engine is used.** Llamia-Verb mostly copies the engine's top move (62–76%). Llamia uses different strategies per task, e.g. querying hypothetical moves for commentary.
- **Cost.** Llamia makes fewer engine calls (1.9 vs. 2.9 per query); inference cost is about the same.
- **Human study** (n=12, rated ≥1700). Players spotted Llamia as a bot in 39% of games (Llamia-Verb: 72%); 72% of commentary judgments preferred Llamia.
- **Beyond chess.** On Go with KataGo, the same method is ~10 points above the text-based control.